

Machine Learning: Evaluating Learning Algorithms

Dr. Oybek Eraliev,

Department of Computer Engineering

Inha University In Tashkent.

Email: oybekeraliev7@gmail.com



Content

- **Introduction to Evaluation**
- **Performance Metrics**
- **Implementation of Performance Metrics**

Introduction to Evaluation

- In machine learning, evaluating the performance of learning algorithms is a crucial step in understanding how well a model generalizes to unseen data.
- The goal is not only to minimize error on the training set but also to ensure the model performs well on new, unseen data.
- Effective evaluation helps identify the strengths, weaknesses, and potential improvements needed in a model.

Introduction to Evaluation

Importance of Evaluating Learning Algorithms

1. **Generalization Ability:** The ultimate goal of a machine learning model is to generalize well on unseen data. Evaluation helps assess whether a model trained on a given dataset will work effectively on new data. Overfitting (performing well on training data but poorly on test data) and underfitting (poor performance on both) are common issues that proper evaluation helps to identify.
2. **Model Selection:** Machine learning often involves selecting the best model from several options. Without an objective evaluation, it's difficult to determine which model performs best. Evaluation metrics guide the selection by providing a standardized way to compare models.

Introduction to Evaluation

Importance of Evaluating Learning Algorithms

3. **Parameter Tuning:** Machine learning models often have hyperparameters that need to be optimized. Evaluation helps assess the effect of different hyperparameter settings and fine-tune them for optimal performance.
4. **Understanding Model Strengths and Weaknesses:** Evaluation enables practitioners to understand in which scenarios their model excels and where it struggles. For example, certain models may work well in high-dimensional spaces but poorly with noisy data, and evaluation helps in identifying such characteristics.

Introduction to Evaluation

Importance of Evaluating Learning Algorithms

5. **Benchmarking:** To measure progress, models need to be evaluated against baselines and benchmarks. Evaluation metrics help compare the performance of the current model with previous models or other competitors' models.

Content

- ~~Introduction to Evaluation~~
- Performance Metrics
- Implementation of Performance Metrics

Performance Metrics

Supervised learning

```
graph TD; A[Supervised learning] --> B[A. Classification problem]; A --> C[B. Regression problem];
```

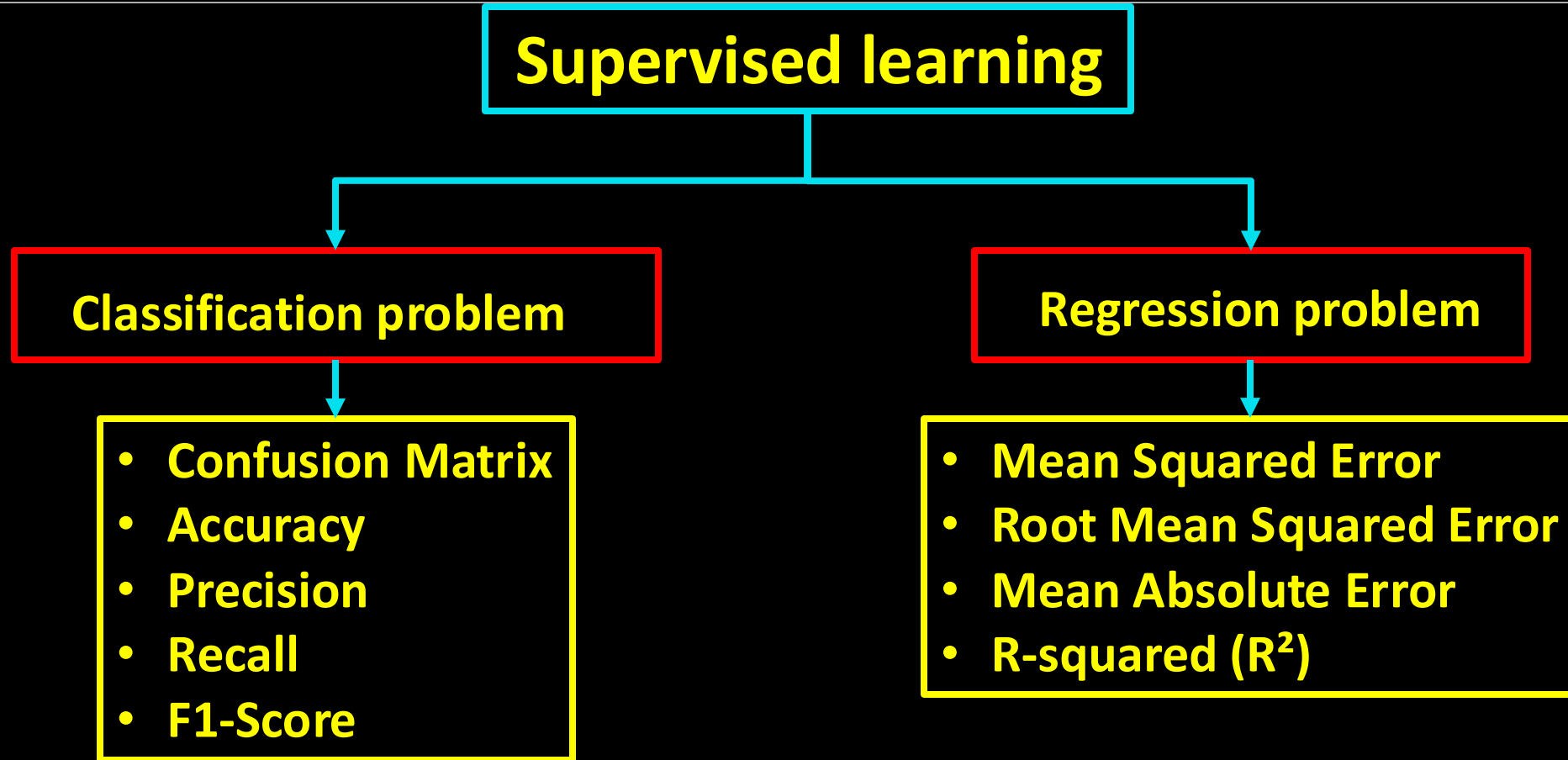
A. Classification problem

- Logistic Regression
- Decision Tree
- Naive Bayes
- K – Nearest Neighbor
- Support Vector Machine

B. Regression problem

- Linear Regression
- Decision Tree
- Support Vector Machine
- Ridge Regression
- Stepwise Regression

Performance Metrics



Performance Metrics

Regression models

- **Mean Squared Error**
- **Root Mean Squared Error**
- **Mean Absolute Error**
- **R-squared (R^2)**

Mean Squared Error (MSE): Measures the average of the squares of the errors between predicted and actual values. It is sensitive to large errors.

$$MSE = \frac{1}{m} \cdot \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)})^2$$

Performance Metrics

Regression models

- Mean Squared Error
- **Root Mean Squared Error**
- Mean Absolute Error
- R-squared (R^2)

Root Mean Squared Error (RMSE): The square root of the MSE, giving error in the same units as the target variable.

$$MSE = \frac{1}{m} \cdot \sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)})^2$$

$$RMSE = \sqrt{MSE}$$

Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- R-squared (R^2)

Mean Absolute Error (MAE): The average of the absolute differences between predicted and actual values. Less sensitive to large errors than MSE.

$$MAE = \frac{1}{m} \cdot \sum_{i=1}^m |h_{\theta}(x)^{(i)} - y^{(i)}|$$

Performance Metrics

Regression models

- Mean Squared Error
- Root Mean Squared Error
- Mean Absolute Error
- **R-squared (R^2)**

R-squared (R^2): Represents the proportion of the variance in the target variable explained by the model. A value close to 1 indicates a good fit.

$$R^2 = 1 - \frac{\sum_{i=1}^m (h_{\theta}(x)^{(i)} - y^{(i)})^2}{\sum_{i=1}^m (h_{\theta}(x)^{(i)} - y_{mean})^2}$$

$$y_{mean} = \sum_{i=1}^m y^{(i)} / m$$

Performance Metrics

Classification problem

- **Confusion Matrix**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**

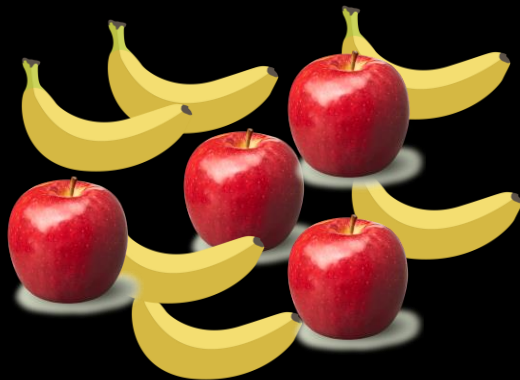
Confusion Matrix: A table that breaks down the predicted versus actual values, offering a detailed look at performance for each class (True Positives, False Positives, True Negatives, and False Negatives).

		Predicted	
		Negative	Positive
Actuals	Negative	True Negatives	False Positives
	Positive	False Negatives	True positives

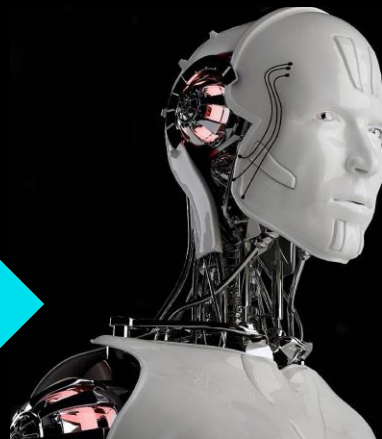
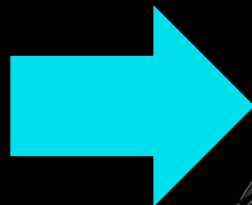
Performance Metrics

Classification problem

- **Confusion Matrix**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**



Test set

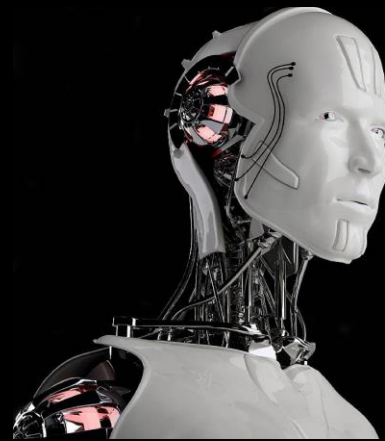


**Model
(Algorithm)**

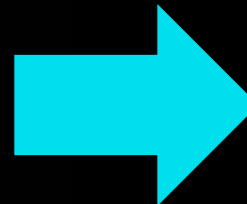
Performance Metrics

Classification problem

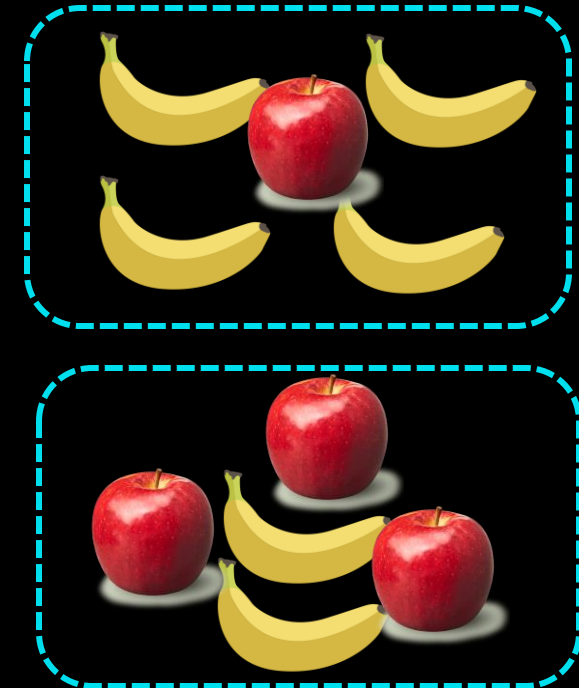
- **Confusion Matrix**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**



**Model
(Algorithm)**



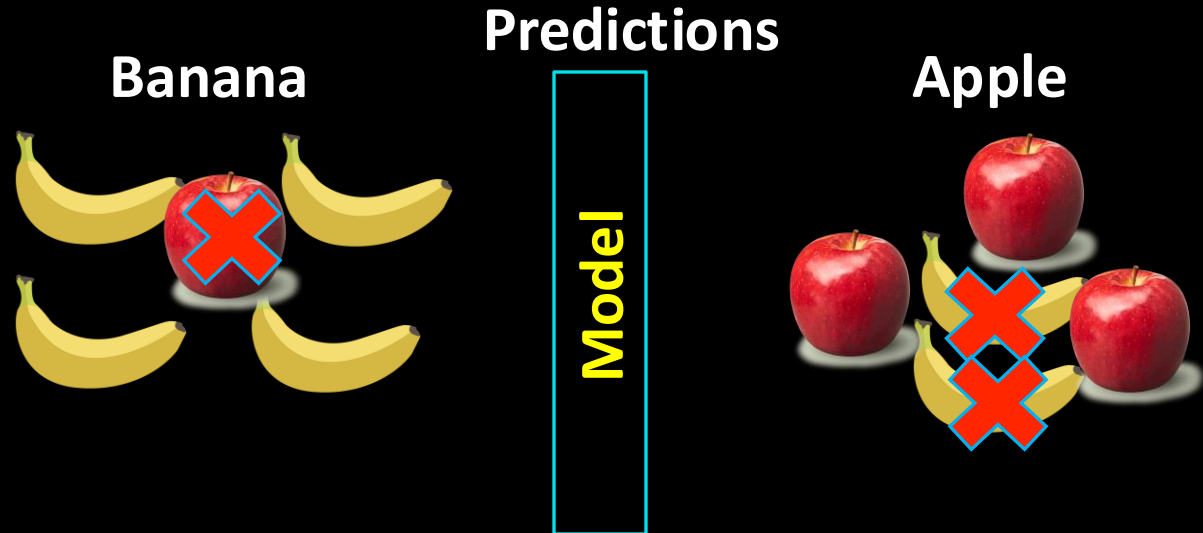
Predictions



Performance Metrics

Classification problem

- **Confusion Matrix**
- Accuracy
- Precision
- Recall
- F1-Score



Performance Metrics

Classification problem

- **Confusion Matrix**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**

Apple = positive

Banana = negative

		Predictions	
		Banana	Apple
Actuals	Banana	 4	 2
	Apple	 1	 3

Performance Metrics

Classification problem

- **Confusion Matrix**
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**

		Predictions		
		Negative	Positive	
Actuals	Negative	4	2	True Negative
	Positive	1	3	False Positive
		False Negative		True Positive

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Accuracy: The proportion of correctly classified instances over the total number of instances.

$$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \cdot 100\%$$

		Negative	Positive
Actuals	Negative	True Negative 4	False Positive 2
	Positive	False Negative 1	True Positive 3

$$Accuracy = \frac{4 + 3}{10} \cdot 100\% = 70\%$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Precision: The proportion of true positive instances among all instances predicted as positive. Precision is critical when false positives are costly.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

		Predictions	
		Negative	Positive
Actuals	Negative	True Negative 4	False Positive 2
	Positive	False Negative 1	True Positive 3

$$\text{Precision} = \frac{3}{3 + 2} = 0.6$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- F1-Score

Recall (Sensitivity): The proportion of true positive instances among all actual positive instances. It's useful when missing a positive instance (false negative) is costly.

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

		Predictions	
		Negative	Positive
Actuals	Negative	True Negative 4	False Positive 2
	Positive	False Negative 1	True Positive 3

$$Recall = \frac{3}{3 + 1} = 0.75$$

Performance Metrics

Classification problem

- Confusion Matrix
- Accuracy
- Precision
- Recall
- **F1-Score**

F1 Score: The harmonic mean of precision and recall. It balances the trade-off between precision and recall and is useful when you need a single measure that accounts for both.

$$\text{Precision} = \frac{3}{3 + 2} = 0.6$$

$$\text{Recall} = \frac{3}{3 + 1} = 0.75$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F1 = 2 \cdot \frac{0.6 \cdot 0.75}{0.6 + 0.75} = 0.667$$

Implementation of Performance Metrics

Regression models

```
import numpy as np

# Predicted values (h(x)) and actual target values (y)
h_x = np.array([11, 19, 28, 35, 48, 60, 66, 78, 91, 99])
y = np.array([10, 20, 30, 40, 40, 60, 70, 80, 90, 100])

# Mean Squared Error (MSE)
mse = np.mean((y - h_x)**2)

# Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# Mean Absolute Error (MAE)
mae = np.mean(np.abs(y - h_x))

# R-squared (R²)
y_mean = np.mean(y)
ss_total = np.sum((y - y_mean)**2)
ss_residual = np.sum((y - h_x)**2)
r_squared = 1 - (ss_residual / ss_total)

# Print the results
print(f"MSE: {mse}")
print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"R-squared: {r_squared}")
```

[1]

Implementation of Performance Metrics

Classification models

```
import numpy as np

h_x_binary = np.array([0, 0, 0, 1, 1, 1, 0, 0, 0])
y_binary = np.array([0, 0, 1, 1, 1, 0, 0, 0, 1])

# Confusion matrix components
true_positive = np.sum((h_x_binary == 1) & (y_binary == 1))
true_negative = np.sum((h_x_binary == 0) & (y_binary == 0))
false_positive = np.sum((h_x_binary == 1) & (y_binary == 0))
false_negative = np.sum((h_x_binary == 0) & (y_binary == 1))

# Accuracy
accuracy = (true_positive + true_negative) / len(y_binary)

# Precision
precision = true_positive / (true_positive + false_positive) if (true_positive + false_positive) > 0 else 0

# Recall (Sensitivity)
recall = true_positive / (true_positive + false_negative) if (true_positive + false_negative) > 0 else 0

# F1 Score
f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

# Confusion matrix
confusion_matrix = np.array([[true_negative, false_positive], [false_negative, true_positive]])

# Print the results
print(f"Predicted values: {h_x_binary}")
print(f"True values: {y_binary}")
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1 Score: {f1_score}")
print("Confusion Matrix:")
print(confusion_matrix)
```

[6]



